CFLP - examen

1. Piece of code (Analiză de cod)

Examenul poate include bucăți de cod din diverse paradigme prezentate în cursuri. Trebuie să fii pregătit să recunoști și să analizezi:

- **C/C++:** Pointeri, aritmetica pointerilor, structuri,.
- **Java:** Clase, moștenire, Socket-uri (Client-Server),.
- **Python:** Comprehensions, liste, tupluri, funcții Lambda,.
- **Lisp/Lambda Calcul:** Reduceri beta, funcții anonime,.
- **Edison:** Concurență (`cobegin` , `when`),.

2

Client Server Model

- Este un sistem distribuit unde serverul gestionează resurse, iar clientul le accesează.
  - Procesele programului server : gestioneaza resursele
  - Procesele programului client : acceseaza resursele gestionate de severe
- Mesajele sunt limitate la maxim o linie
- Serverul :
  - Trebuie sa fie primul pornit
  - A fost dezvoltat ˆın unitatea de compilare Server.java
- Clientul:
  - Trimite o cerere
  - Asteapta un raspuns
  - Trimite comanda STOP
  - A fost dezvoltat in unitatea de compilare Client.java
- Clientul se conecteaza la server prin instantierea unui obiect de tip conexiune soclu(Socket)
- Primul mesaj al clientului catre server contine soclul clientului

```
// Client.java
import java.net.*;
import java.io.*;
class Client
{
    public static void main(String[] args) throws IOException
    {
        Socket cs=null;
        BufferedReader is=null; DataOutputStream os=null;
```

```
        try
        {
            cs=new Socket("localhost",5678);
            is=new BufferedReader(new
InputStreamReader(cs.getInputStream()));
            os=new DataOutputStream(cs.getOutputStream());
        }
        catch(UnknownHostException e)
        {
            System.out.println("No such host");
        }

        // Continuare logică (Imaginea 2)
        BufferedReader stdin=
            new BufferedReader(new InputStreamReader(System.in));
        String line;
        for(;;)
        {
            line=stdin.readLine()+"\n";
            os.writeBytes(line);
            System.out.println("Transmission:\t"+line);
            if(line.equals("STOP\n")) break;
            line=is.readLine();
            System.out.println("Receiving:\t"+line);
        }
        System.out.println("READY");
        cs.close(); is.close(); os.close();
    }
}
```

```
// Server.java
import java.net.*;
import java.io.*;

class Server
{
    public static void main(String[] args) throws IOException
    {
        ServerSocket ss=null; Socket cs=null;
        BufferedReader is=null; DataOutputStream os=null;
        try
        {
            ss=new ServerSocket(5678);
            System.out.println("The server is running!");
            cs=ss.accept();
            is=new BufferedReader(new
InputStreamReader(cs.getInputStream()));
            os=new DataOutputStream(cs.getOutputStream());
```

```
            BufferedReader stdin=
                new BufferedReader(new InputStreamReader(System.in));
            String line;
            for(;;)
            {
                line=is.readLine();
                System.out.println("Receiving:\t"+line);
                if(line.equals("STOP")) break;
                line=stdin.readLine()+"\n";
                os.writeBytes(line);
            }
        }
        finally
        {
            cs.close(); ss.close();
            is.close(); os.close();
        }
    }
}
```

How threads work

- În surse, conceptul este tratat prin prisma **concurenței** și a proceselor logice.
- **Concurența** implică "time-slicing" (partajarea timpului) pe un singur procesor, creând iluzia de paralelism,.
- Limbajul **Edison** exemplifică lansarea execuției paralele prin instrucțiunea `cobegin` (unde fiecare ramură este un proces) și sincronizarea prin regiuni critice condiționate folosind `when`,.
- Problema clasică prezentată este **Problema Filozofilor**, care ilustrează excluderea mutuală și riscul de înfometare (starvation).

3

Metasimboluri EBNF

::= înseamnă "definit ca"
| înseamnă "sau"
< și > sunt utilizate pentru neterminale
[și] sunt utilizate pentru secvențe opționale
{ și } sunt utilizate pentru secvențe ce se repetă de zero sau mai multe ori

Write an AST

- Un AST este rezultatul analizei sintactice. Trebuie să știi să desenezi arborele de derivare pornind de la o gramatică BNF.

- Rădăcina este simbolul de start, nodurile interne sunt neterminale, iar frunzele sunt terminale (atomi).

• **Unde plasăm operatorii (ex: adunarea +**)?**

◦ În AST, operatorul `+` devine un **nod intern** (părinte). El leagă cei doi operanzi (fiii), stabilind ordinea de evaluare.

• **Exemplu:** Pentru `a := 2 * (x + 3)`:

◦ Rădăcina este `:=`.

◦ Ramura stângă este `a`.

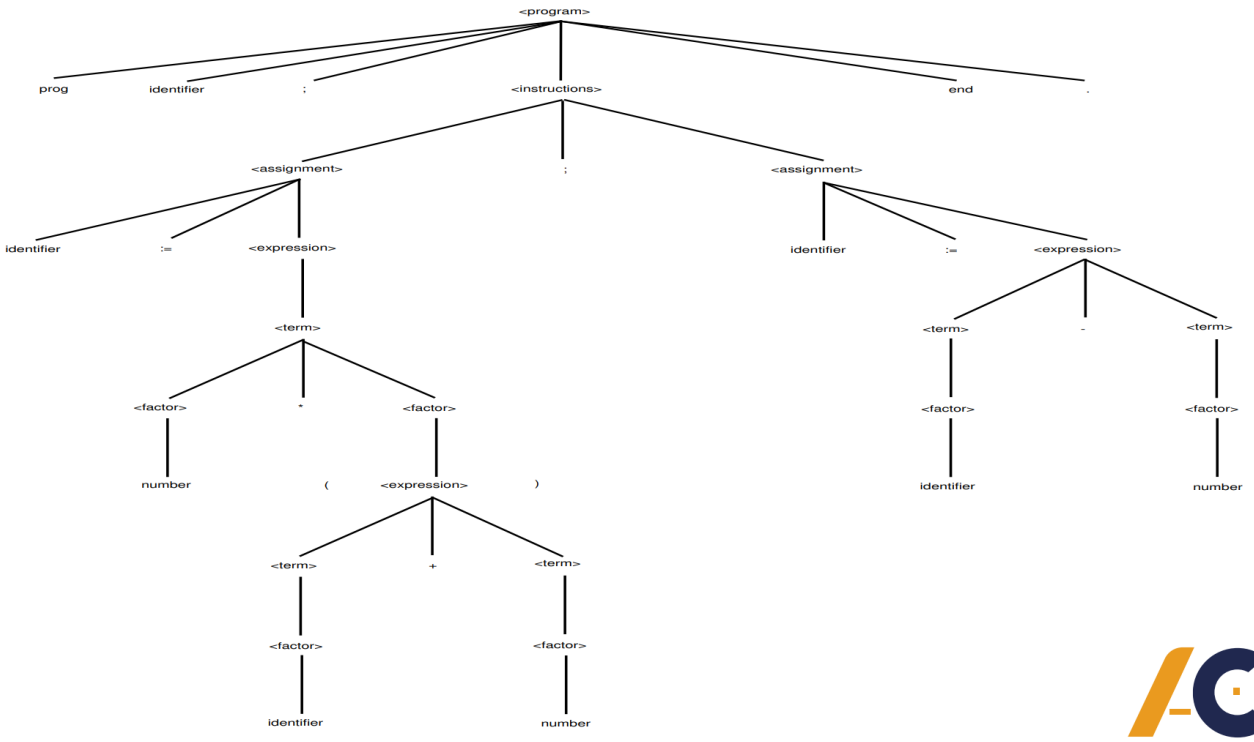
◦ Ramura dreaptă are ca rădăcină `*`, care are un fiu `2` și un fiu `+` (care la rândul lui are fiii `x` și `3`).

Notă despre Expresii Regulate (Regex): Acestea se folosesc strict la **analiza lexicală** (înainte de AST). Aici, `+` este tratat fie ca un simplu caracter (simbol), fie ca un operator de repetiție (unu-sau-mai-multe), **nu** ca o operație matematică

Exemplu de gramatica

```
<expression>   ::= <term> { +|- <term> }
<term>         ::= <factor> { *|: <factor> }
<factor>       ::= number | identifier | ( <expression> )
<assignment>   ::= identifier := <expression>
<instructions> ::= <assignment> { ; <assignment> }
<program>      ::= prog identifier; <instructions> end.

prog example;
a:=2*(x+3);
b:=a-1
end.
```



Operatori pentru expresii regulate

Operatori de bază

1. Reuniune (Selection): (e1)|(e2)
   - Rezultat: mulțimea S1 ∪ S2
   - Semnificație: "sau" (e1 sau e2)
2. Produs (Concatenare): (e1)(e2)
   - Rezultat: mulțimea S1S2
   - Semnificație: e1 urmat imediat de e2
3. Steaua Kleene (Iterație): (e)*
   - Rezultat: mulțimea S*
   - Semnificație: zero sau mai multe apariții

Operatori extinși

4. Plus (Unul sau mai mulți): e+
   - Echivalență: e+ este echivalent cu ee*
   - Semnificație: cel puțin o apariție
5. Semnul întrebării (Zero sau unul): e?
   - Semnificație: apariție opțională

Clase de caractere

- Notație explicită: [c1c2c3c4] este echivalent cu c1|c2|c3|c4
- Intervale: [a-z] este echivalent cu a|b|...|z

Reguli de precedență și asociativitate

- Asociativitate: Toți operatorii sunt asociativi la stânga.
- Prioritate (de la mare la mică):
  1. Stea (*), Plus (+), Semnul întrebării (?)
  2. Produs / Concatenare
  3. Reuniune (|)

```
// Definiții de bază
cifra            -> [0-9]
litera           -> [A-Z,a-z]

// 1. Identificator
// (O literă urmată de oricâte litere sau cifre)
identificator    -> litera(litera|cifra)*

// 2. Număr
// Helper pentru șir de cifre (una sau mai multe)
cifre            -> cifra+

// Componenta opțională pentru exponent (ex: E+10, e-5)
exponent         -> ((E|e)(+|-)?cifre)?

// Componenta opțională pentru zecimale
parte_fractionara -> (.cifre)?

// Definiția finală a numărului
numar            -> cifre parte_fractionara exponent
```

Semantic Action on Grammar (Gramatici Atributate / DDS)

Aici se definește **sensul** și se efectuează calculele logice folosind **Definiții Dirijate de Sintaxă (DDS)**. Se atașează atribute simbolurilor gramaticale,.

• **Mecanism:** Regulile semantice sunt asociate producțiilor gramaticale. Valorile sunt calculate în timpul parcurgerii arborelui (adnotare).

• **Unde plasăm adunarea?**

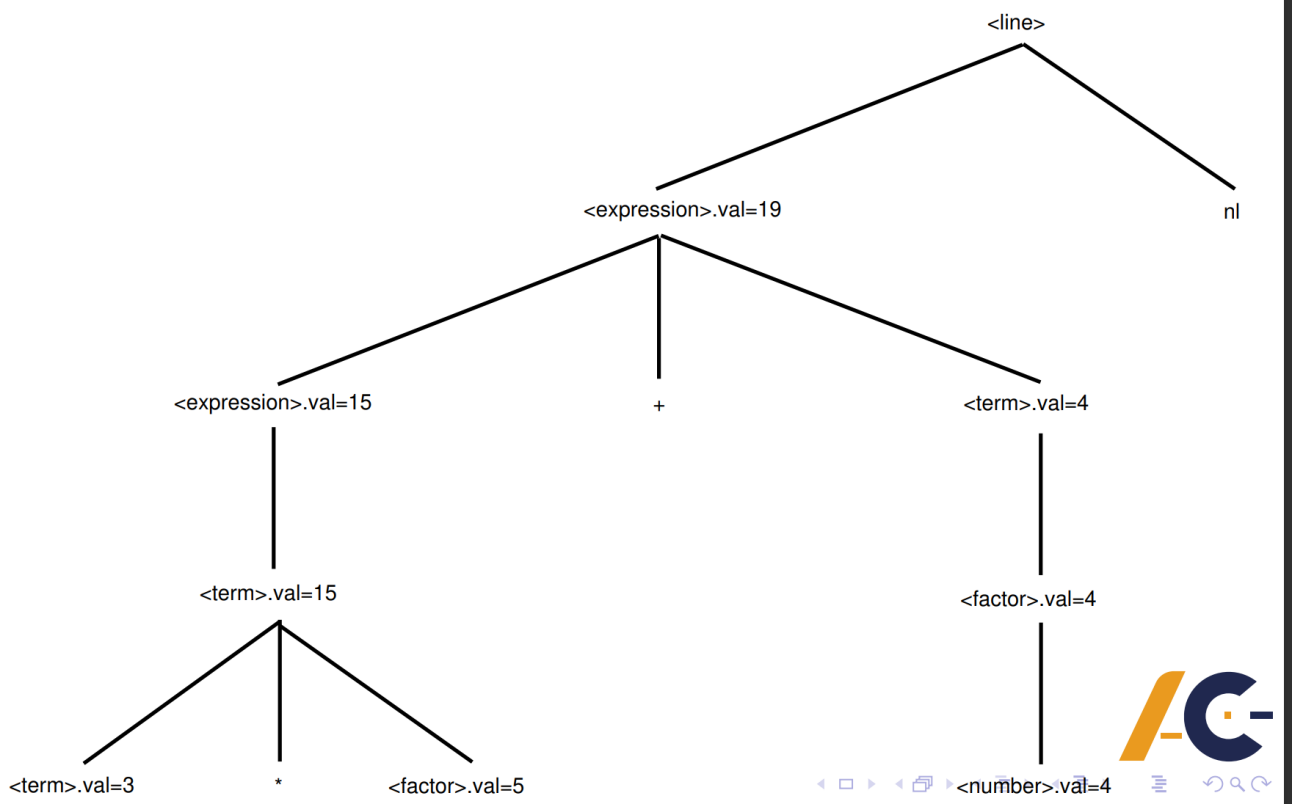
- Aici are loc calculul efectiv sau definirea lui logică.
- **Regulă:** `<expression>.val := <expression1>.val + <term>.val`.
- Atributul `.val` sintetizează rezultatul de la nivelurile inferioare către cele superioare.

• **Exemplu:** Pentru `<term> ::= <term1> * <factor>`, regula este `<term>.val := <term1>.val * <factor>.val`.

```
<line>::=<expression>nl
<expression>::=<expression>+<term>|<term>
<term>::=<term>*<factor>|<factor>
<factor>::=(<expression>)|number
```

- definiția asociază la fiecare neterminal $< expression >$, $< term >$, $< factor >$ câte un atribut întreg cu numele val
- Pentru fiecare producție calculăm:
  - atributul val asociat cu neterminalul din partea stangă
  - bazându-ne pe valorile atributelor val ale neterminalelor din partea dreaptă

Grammar production	Semantic rules
<line>::=<expression>nl	print(<expression>.val)
<expression>::=<expression1>+<term>	<expression.val>:=<expression1>.val+<term>.val
<expression>::=<term>	<expression>.val:=<term>.val
<term>::=<term1>*<factor>	<term>.val:=<term1>.val*<factor>.val
<term>::=<factor>	<term>.val:=<factor>.val
<factor>::=(<expression>)	<factor>.val:=<expression>.val
<factor>::=number	<factor>.val:=number.lexval



Translate Code (High level to Low level)

Aceasta reprezintă faza de **generare a codului intermediar** (pentru o mașină abstractă), care are loc după analiza semantică.

• **Codul cu 3 adrese:** Este o secvență de instrucțiuni simple de forma `rezultat := operand1 op operand2`. Compilatorul trebuie să genereze variabile temporare (`temp1`, `temp2`) pentru rezultatele parțiale.

• **Unde plasăm adunarea?**

- Operatorul `+` devine o instrucțiune explicită în secvența liniară de cod.

• **Exemplu:** `a := b + c * 10` se traduce astfel:

1. `temp1 := inttoreal(10)` (conversie de tip, dacă e necesar)
2. `temp2 := c * temp1`
3. `temp3 := b + temp2` (Aici are loc adunarea la nivel scăzut)
4. `a := temp3`

Generate Post Condition & Weakest Precondition (Semantica Axiomatică)

Această metodă verifică corectitudinea programelor folosind logică matematică (tripletele lui Hoare `{Precondiție} Instrucțiune {Postcondiție}`).

• **Weakest Precondition (W):** Cea mai slabă (cea mai generală) precondiție necesară pentru ca executând `S` să ajungem în starea `P`. Funcția este `axsem(S, P) = W`.

• **Regula pentru atribuire (**x := E**):**

◦ Pentru a afla precondiția, se înlocuiesc toate aparițiile variabilei `x` cu expresia `E` în postcondiția `P`.

◦ Formula: `axsem(x := E, P) = P[x -> E]`.

• **Exemplu:**

◦ Instrucțiune: `x := x + 3`

◦ Postcondiție cerută: `P: x > 8`

◦ **Calcul:** Înlocuim `x` cu `x + 3` în `x > 8`.

◦ **Rezultat (Weakest Precondition):** `x + 3 > 8` (ceea ce implică `x > 5`)

Notatie:

```
Q {S} P
```

Exemplu:

```
S: x=y+1 (intregi)
P: x>0
Q: y=3
y=3 {x=y+1} x>0
-------------
Q: y>-1
y>-1 {x=y+1} x>0
```

4

Translation vs Compilation & Phases of Compilation

- **Diferența:**
  - **Compilarea (Translatarea):** Programul sursă este tradus complet în cod mașină (sau intermediar) *înainte* de execuție. Este mai rapidă la execuție, dar ocupă mai multă memorie,.
  - **Interpretarea:** Instrucțiunile sunt decodificate și executate pas cu pas de un interpretor. Este mai lentă, dar mai portabilă și ocupă mai puțină memorie,.

    - Soluția hibridă (Java/C#): Compilare în bytecode/MSIL, apoi interpretare sau JIT (Just-In-Time).
- **Fazele Compilării:**
  1. **Analiza Lexicală:** Grupează caracterele în atomi (tokeni) (ex: `id`, `:=`, `num`).
  2. **Analiza Sintactică:** Verifică structura gramaticală și construiește arborele sintactic.
  3. **Analiza Semantică:** Verifică tipurile, vizibilitatea variabilelor.
  4. **Generarea Codului Intermediar:** Cod abstract (ex: cod cu 3 adrese).
  5. **Optimizarea Codului:** Eliminarea redundanțelor.
  6. **Generarea Codului Obiect:** Traducerea finală în cod mașină specific.

5 - code, ask what the value of a variable

Tipuri de legari

- legarea statica
  - inainte de executie
  - nu poate fi schimbata ulterior
- legarea dinamica
  - in momentul executiei
  - poate fi schimbata ulterior

Trebuie să determini valoarea unei variabile într-un cod dat, cunoscând mecanismul de legare.

- **Domeniul de vizibilitate (Scoping):**
  - **Static (Lexical):** Variabila este căutată în blocul curent, apoi în blocurile părinte din *textul sursă* (Pascal, C, Java).
  - **Dinamic:** Variabila este căutată pe *stiva de apeluri* (calea de execuție). Folosit în Lisp vechi sau Perl.
  - *Exemplu:* Dacă funcția `f` folosește `x` global, iar `f` este apelată din `g` care definește un `x` local: în scop static `f` vede `x`-ul global; în scop dinamic `f` vede `x`-ul din `g`,.

6 - code, compute the values of variables + mechanism is known

Transmiterea parametrilor

- **Transmiterea Parametrilor:**
  - **Prin Valoare:** Modificările nu se văd în exterior (C, Java primitive, Pascal implicit).
  - **Prin Referință/Adresă:** Modificările afectează variabila originală (Pascal `var`, C++ `&`, C# `ref`).

- **Prin Nume:** Parametrul este re-evaluat la fiecare utilizare (Algol, Simula) - poate duce la efecte complexe dacă indicii tablourilor se schimbă.

1. Transmiterea Parametrilor: Valoare vs. Referință (Exercițiu de calcul)

Pentru a înțelege ce valori au variabilele la final, trebuie să analizăm diferența fundamentală dintre cele două mecanisme, așa cum sunt definite în surse:

- **Prin Valoare (Value):** Parametrul formal este o **copie** a celui actual. Modificările din funcție **nu** sunt vizibile în exterior.
- **Prin Referință/Adresă (Reference):** Se transmite **adresa** de memorie. Orice modificare a parametrului formal afectează direct variabila originală din exterior.

Exemplu de calcul (Scenariu de Examen): Să considerăm un cod ipotetic (bazat pe sintaxa C++ din sursa și Pascal din):

```
// Definim funcția cu doi parametri:
// 'val' este transmis prin VALOARE (copie)
// 'ref' este transmis prin REFERINȚĂ (adresă - marcat cu &)
void calculeaza(int val, int &ref) {
    val = val + 10;  // Modifică doar copia locală
    ref = ref + 10;  // Modifică variabila originală de la acea adresă
}

int main() {
    int a = 5;
    int b = 5;

    // Apelăm funcția
    calculeaza(a, b);

    // ÎNTREBARE: Ce valori au a și b acum?
}
```

Răspuns și Explicație:

- **La final, `a` are valoarea 5.**
  - *De ce?* Deoarece `a` a fost transmis prin **valoare**. Funcția a primit doar copia valorii (5), a făcut-o 15, dar variabila originală `a` din `main` a rămas neatinsă.
- **La final, `b` are valoarea 15.**
  - *De ce?* Deoarece `b` a fost transmis prin **referință**. Funcția a primit adresa lui `b`. Instrucțiunea `ref = ref + 10` a scris direct în memoria unde se află `b`.

---

2. Efecte Secundare (Side Effects)

Efectele secundare apar atunci când un subprogram modifică starea sistemului în afara mediului său local. Sursele prezintă acest concept în câteva contexte cheie:

- **Modificarea parametrilor mutabili (Python):**
  - În Python, listele sunt obiecte mutabile. Dacă o listă este transmisă unei funcții și este modificată folosind operatori precum `+=` sau metode, lista originală se modifică. Acesta este un efect secundar.
  - *Exemplu:* Dacă funcția face `cities += ["Sibiu"]`, lista originală din afara funcției va conține și "Sibiu".
  - *Evitare:* Pentru a evita efectele secundare, se poate transmite o copie a listei folosind slicing: `function(lista[:])`.
- **Transmiterea prin Nume (Call by Name):**
  - Acest mecanism (folosit în Algol 60) poate genera efecte secundare complexe și greu de înțeles.
  - *Exemplu critic:* Într-o funcție de interschimbare `swap(i, t[i])`, re-evaluarea numelor la fiecare pas poate duce la rezultate eronate (ex: `i` se schimbă înainte să se acceseze `t[i]`), modificând indici greșiți în tablou.

---

3. Subprograme Generice (Cod)

Un subprogram generic permite scrierea unui algoritm o singură dată, care să funcționeze pentru orice tip de date (polimorfism parametric). Sursele oferă exemple în Ada și C++.

Exemplu C++ (Template) Acesta este exemplul clasic de sortare generică prezentat în curs,:

```
// Definiția șablonului (Template definition)
// T este un tip generic care va fi înlocuit la compilare (ex: int, double)
template <class T> void sort(T *array, int size)
{
    register int i, j;
    T temp; // Variabilă temporară de tipul generic T

    for(i = 1; i < size; i++)
    {
        for(j = size - 1; j >= i; j--)
        {
            // Comparația funcționează dacă tipul T suportă operatorul >
            if(array[j-1] > array[j])
            {
                // Interschimbarea valorilor
                temp = array[j-1];
```

```
                array[j-1] = array[j];
                array[j] = temp;
            }
        }
    }
}

// Utilizare (Instanțiere):
// sort(arrayofint, 10);    -> T devine int
// sort(arrayofdouble, 20); -> T devine double
```

Alternativa Ada (pentru completitudine) În Ada, genericitatea este specificată explicit prin clauza `generic`:

```
generic
    type tip_el is private; -- tip generic
    type vec is array (integer range< >) of tip_el;
    zero: tip_el;           -- element neutru generic
    with function "+"(x,y:tip_el) return tip_el; -- operator generic
function apply(v:vec) return tip_el;
```

7

1. Tipuri de Date Predefinite pe Limbaje

Sursele clasifică tipurile de date în funcție de reprezentarea lor hardware și suportul în limbaj.

- **C și C++:**
  - **Numerice:** `char`, `short`, `int`, `long`, `float`, `double`, `long double`.
  - **Logice:** `bool` (în C++ și C99).
  - **Caractere:** `char` (folosit și pentru ASCII).
  - **Pointeri:** Generici (`void*`) sau tipizați (`int*`); în C++ există și pointeri inteligenți (`unique_ptr`, `shared_ptr`),.
  - **Reuniuni:** `union` (mai multe variabile împart aceeași memorie).
- **Java:**
  - **Numerice:** `byte`, `short`, `int`, `long`, `float`, `double`.
  - **Logice:** `boolean` (cu valori `true`, `false`).
  - **Caractere:** `char` (Unicode).
  - **Tipuri Referință:** Clase, tablouri. `String` este un tip imuabil dedicat.
- **C#:**
  - **Numerice:** Include variante fără semn (`unsigned`) și tipul zecimal precis: `short`, `ushort`, `int`, `uint`, `long`, `ulong`, `float`, `double`, **`decimal`**.
  - **Logice:** `bool`.

    - **Enumerări:** `enum` (ex: `enum Level {LOW, HIGH}`).
- **Python:**
    - **Numerice:** `int`, `float`, `complex` (numere complexe suportate nativ).
    - **Secvențiale:** `list` (mutabilă), `tuple` (imuabilă), `range`.
    - **Dicționare:** `{key: value}`.
    - **Mulțimi:** `set` (ex: `x = {"apple", "banana"}`).
- **JavaScript:**
    - Este prezentat ca un limbaj dinamic unde obiectele funcționează ca **dicționare**.
    - Se pot defini obiecte literal: `var dict = {FirstName: "Chris", "one": 1}`.
    - Proprietățile pot fi adăugate dinamic: `dict["Age"] = 42`.

2. Crearea Structurilor de Date (Create Data Structures)

Limbajele oferă mecanisme specifice pentru a agrega datele primitive în structuri complexe:

A. Produsul Cartezian (Structuri/Înregistrări) Aceasta este metoda de a grupa elemente de tipuri **diferite**.

- **C:** Folosește `struct`.
  ```
  struct node { info_type info; struct node *left, *right; }
  ```
- **Java/C#:** Folosesc `class` cu modificatori de acces (`private`, `public`).
  ```
  class Node { private InfoType info; private Node left, right; }
  ```
- **Pascal/Ada:** Folosesc `record`.

B. Proiecția Finită (Tablouri/Arrays) Grupează elemente de **același** tip, accesibile prin index.

- **Static (C/Pascal):** Dimensiune fixată la compilare (ex: `char a`).
- **Dinamic:** Pointeri în C, obiecte în Java/C#, sau liste în Python care permit felierea (slicing: `list[2:5]`).

C. Recurența (Structuri Recursive) Pentru a crea liste înlănțuite sau arbori, structura trebuie să conțină o referință către ea însăși.

- În **C/C++**, acest lucru obligă utilizarea **pointerilor** (`struct node *left`).
- În **Java/C#**, referințele sunt implicite pentru obiecte (`Node left`).

---

3. Implementarea unei Mulțimi (Set)

În cerința ta ("o multime de implementat"), sursele explică atât cum se folosesc mulțimile native, cât și cum se pot implementa manual dacă limbajul nu le are.

Ce este un tip Mulțime? O variabilă de tip mulțime poate reține orice submulțime de valori ale unui tip de bază, iar operațiile principale sunt reuniunea, intersecția, diferența și testul de apartenență,.

Varianta 1: Utilizarea tipurilor native (Python) Dacă folosești Python, implementarea este directă:

```
x = {"apple", "banana", "cherry"} # Definire
# Operatiile de reuniune/intersectie sunt native
```

Varianta 2: Implementarea manuală (C, Java, C++ etc.) Dacă trebuie să "creezi" structura de date `Set` de la zero, sursa sugerează următoarele strategii de implementare:

1. **Tablouri de elemente booleene (Bit Vector):**
   - Dacă universul elementelor este mic și cunoscut (ex: numere de la 0 la 100), faci un array `bool exists`.
   - `add(x)` devine `exists[x] = true`.
   - `contains(x)` este doar `return exists[x]`.
2. **Tablouri de biți:**
   - Similar cu mai sus, dar eficient din punct de vedere al memoriei (1 bit per element).
3. **Liste:**
   - Stochezi elementele într-o listă înlănțuită sau dinamică.
   - La `add(x)`, parcurgi lista pentru a verifica dacă `x` există deja (pentru a evita duplicatele).
4. **Arbori:**
   - Pentru performanță mai bună la căutare (`O(log n)`), poți implementa mulțimea folosind un **Arbore Binar de Căutare** (BST), unde fiecare nod este unic.

Exemplu conceptual de structură pentru Mulțime (bazat pe reuniuni variabile în C): Poți folosi și `union` pentru a stoca date polimorfice într-o structură, deși sursa avertizează că reuniunile variabile sunt periculoase în C, deoarece verificarea tipului cade în sarcina programatorului,.

8

1. Sistemul de Tipuri în C (Tipizare Statică și Explicită)

În limbajul C, tipurile sunt legate de variabile la compilare, iar dimensiunile și structura memoriei sunt strict definite.

Tipuri Predefinite și Numerice:

- **Caractere:** `char` ocupă un octet.

- **Întregi:** `int` reprezintă mulțimea numerelor întregi pe mașina gazdă, având variantele `short` (16 biți) și `long` (minim 32 biți). Acestea pot fi `signed` (cu semn) sau `unsigned` (fără semn).
- **Reale:** Există tipurile `float` și `double`, definite în biblioteci precum `<float.h>`.
- **Enumerări:** `enum` permite definirea unor constante simbolice (ex: `enum boolean {NO, YES}`).

Tablouri (Arrays):

- Sunt statice, dimensiunea lor trebuie cunoscută la compilare (`v`).
- Tablourile multidimensionale sunt vectori de vectori (`int mat`), iar accesul se face `mat[i][j]`.
- Numele unui tablou este, de fapt, adresa primului element.

Structuri și Uniuni:

- **Structuri (`struct`):** Implementează produsul cartezian; câmpurile sunt accesate prin operatorul de calificare (`.`).
- **Uniuni (`union`):** Permit stocarea mai multor tipuri în aceeași zonă de memorie (reuniuni variabile), dar la un moment dat o singură reprezentare este validă.
    - *Atenție:* La uniuni nu se face verificare de tip, responsabilitatea corectitudinii revenind programatorului.

Pointeri:

- Pointerii stochează adrese de memorie și sunt puternic tipizați (`int *p`), deși există și tipul generic `void *`.
- Permit aritmetica pointerilor (adunare, scădere) și sunt esențiali pentru structuri recursive (liste, arbori).
- O structură recursivă trebuie să conțină pointeri către ea însăși, nu instanțe directe.

Echivalența de tipuri:

- Se bazează pe echivalența structurală, cu excepția `struct` și `union` care sunt considerate tipuri diferite chiar dacă au aceeași structură internă.

---

2. Sistemul de Tipuri în Python (Tipizare Dinamică)

În Python, verificarea tipurilor se face la rulare. Variabilele pot referi valori de tipuri diferite pe parcursul execuției.

Tipuri Predefinite:

- **Numerice:** `int`, `float` și, spre deosebire de C, `complex` (numere complexe suportate nativ, ex: `1j`).
- **Text:** `str` (șiruri de caractere), care sunt imuabile.
- **Boolean:** `bool` cu valorile `True` și `False`.
- **Secvențe:** `list`, `tuple`, `range`.
- **Mapări și Mulțimi:** `dict` (dicționare), `set` (mulțimi), `frozenset`.
- **Tipuri binare:** `bytes`, `bytearray`, `memoryview`.
- **Tipul vid:** `NoneType` (valoarea `None`).

Mecanismul Dinamic:

- Tipul este asociat valorii, nu numelui variabilei. O variabilă `x` poate fi inițializată cu un întreg (`x=5`), apoi poate primi un șir (`x="Hello"`) sau o listă.
- Conversiile se fac explicit prin constructori precum `int()`, `float()`, `str()`.

Structuri de Date Specifice:

- **Șiruri (`str`):** Pot fi delimitate de ghilimele simple, duble sau triple (pentru text pe mai multe linii). Suportă operații de "feliere" (slicing) precum `b[2:5]` și metode specifice (`upper`, `strip`, `replace`).
- **Liste (`list`):**
  - Sunt eterogene (pot conține tipuri diferite: `["abc", 34, True]`).
  - Sunt mutabile: permit adăugare (`append`, `insert`), ștergere (`remove`, `pop`, `del`) și modificare prin index.
  - Suportă slicing și iterare (`for x in thislist`).
- **Tupluri (`tuple`):** Sunt secvențe imuabile (nu pot fi modificate după creare) și permit "despachetarea" valorilor în variabile individuale (`(green, yellow) = fruits`).
- **Mulțimi (`set`):** Colecții neordonate de elemente unice, suportând operații matematice: reuniune (`union`), intersecție (`intersection`), diferență (`difference`).
- **Dicționare (`dict`):** Colecții de perechi cheie-valoare (`{"brand": "Ford"}`), care permit accesul și modificarea valorilor pe baza cheii.

Booleeni:

- În Python, anumite valori sunt evaluate automat ca `False`: `0`, `None`, șirurile vide `""`, listele/tuplurile/dicționarele vide `[]`, `()`, `{}`. Orice altceva este, de regulă, `True`.

9

10 - OOP Concepts & Implementation

- **Concepte:**
  - **Abstractizare:** Modelarea realității prin clase.
  - **Încapsulare:** Ascunderea datelor (`private`) și acces prin metode (`public`).

    - **Moștenire:** Extinderea claselor (`extends` , `:`).
    - **Polimorfism:** Capacitatea obiectelor de a lua mai multe forme (dinamic prin suprascriere/overriding).
- **Implementare:**
    - **C++:** Clase, secțiuni `public/private` , moștenire (`: public Base`), pointeri pentru legare dinamică.
    - **Java:** `class` , `extends` , `super()` , totul este obiect (exceptând primitivele), interfețe,.

Rezumat

Bazat pe sursa furnizată (**Cursul 10 - Limbaje de Programare Orientate pe Obiecte**), iată o descriere detaliată a fiecărui capitol. Cursul este structurat pe patru părți principale, trecând de la concepte teoretice la implementări specifice în C++ și Java.

1. Concepte Fundamentale (Concepts)

Acest capitol introduce paradigma Orientată pe Obiecte (OOP), definită ca o metodă de organizare a codului folosind obiecte, menită să reflecte entități din lumea reală.

- **Obiectul:** Este conceptul central, reprezentând o instanțiere a unei clase. Un obiect are o **stare** (definită prin variabile locale/atribute) și un **comportament** (set de metode). Starea este implicit inaccesibilă din exterior (încapsulare), interacțiunea realizându-se doar prin metode publice.
- **Clasa:** Este un constructor de tip care descrie variabilele și metodele obiectelor. Este conceptul fundamental în programarea bazată pe obiecte.
- **Cei 4 Piloni ai OOP:**
    1. **Încapsularea (Encapsulation):** Gruparea datelor și a metodelor într-o singură unitate (clasa) și restricționarea accesului direct la componentele interne (prin modificatori precum `private` , `protected` , `public`),. Avantajele includ modularitatea și protecția stării interne.
    2. **Moștenirea (Inheritance):** Permite definirea unor clase noi (subclase) pe baza unora existente (superclase), preluând starea și comportamentul acestora. Este un mecanism de **specializare** și reutilizare a codului.
    3. **Polimorfismul (Polymorphism):** Abilitatea obiectelor de a lua forme multiple. Permite utilizarea aceleiași interfețe pentru tipuri diferite de obiecte. Poate fi **static** (la compilare, ex: supraîncărcarea funcțiilor) sau **dinamic** (la execuție, ex: suprascrierea metodelor),.
    4. **Abstractizarea:** Identificarea obiectelor din lumea reală și crearea unui model software pentru acestea,.

2. Programare Orientată pe Obiecte în C++

Acest capitol exemplifică conceptele folosind limbajul C++, concentrându-se pe moștenire, pointeri și legare dinamică, prin implementarea unei structuri de date de tip **Stivă (Stack)**.

- **Clasa de Bază (`Stack`):**
  - Este definită cu atribute private (`nr_max` , pointerul `*tab_st` pentru tablou, indexul `ind`) și metode publice (`push` , `pop` , `top`).
  - Constructorul alocă manual memorie (`new int[...]`), specific C++.
- **Moștenirea Publică:**
  - Se definește clasa `StackSpec` care moștenește `Stack` (`class StackSpec : public Stack`).
  - Aceasta extinde funcționalitatea adăugând metoda `push_2` (adaugă două elemente).
- **Specializare în adâncime:**
  - Clasa `StackSpecSpec` moștenește `StackSpec` și adaugă metoda `under_top` (acces la penultimul element),.
  - Exemplul ilustrează restricțiile de acces: metodele din subclase pot accesa membrii `protected` din clasa de bază, dar `main` -ul are restricții,.
- **Legarea Dinamică (Dynamic Binding):**
  - Se demonstrează folosind un pointer la clasa de bază (`Stack *pst`) care este instanțiat cu obiecte derivate (`new StackSpecSpec`).
  - Apelul metodelor prin pointer se rezolvă la execuție în funcție de tipul real al obiectului (polimorfism dinamic).

3. Programare Orientată pe Obiecte în Java

Acest capitol prezintă filozofia Java comparativ cu C++ și oferă un exemplu complex cu forme geometrice.

- **Aspecte Generale și Istoric:**
  - Lansat de Sun în anii '90, inițial pentru dispozitive electronice, apoi adaptat pentru Web datorită portabilității (Bytecode + Virtual Machine),.
  - **Diferențe față de C++:** Nu permite pointeri, nu are dealocare explicită a memoriei (are Garbage Collection), și nu suportă moștenire multiplă (din motive de securitate și simplitate).
  - Este inspirat din SmallTalk (totul este obiect, mai puțin tipurile primitive) și Eiffel,.
- **Exemplul `Circle` (Cercul):**
  - **Clasa `Circle` :** Definește atribute (`x` , `y` , `r`), variabila statică `num_circles` (care numără instanțele) și constructori care se apelează între ei folosind `this(...)` ,.
  - **Moștenirea (`GraphicCircle`):** Extinde `Circle` adăugând culori (`outline` , `fill`) și metoda `draw` . Folosește `super(...)` pentru a apela constructorul părintelui.
  - **Polimorfism și Redefinire (`GraphicCircleSmart`):** O a doua subclasă care redefinește metoda `draw` pentru a desena linii suplimentare peste cerc.
- **Particularități Java evidențiate:**

- Tablourile sunt tratate ca obiecte și au proprietatea `.length`.
- Toate metodele sunt virtuale implicit (legare dinamică), spre deosebire de C++.
- Se folosește casting-ul explicit (`(GraphicCircleSmart)tabCircle`) pentru a accesa metode specifice unei subclase dintr-o referință de tip părinte.
- Există biblioteci grafice bogate (AWT, Swing) menționate în contextul desenării.

4. Bibliografie

Ultimul capitol listează sursele academice de referință pentru curs, incluzând lucrări de:

- Horia Ciocârlie (Universul limbajelor de programare).
- Carlo Ghezzi și Mehdi Jazayeri.
- Donald Knuth (Arta programării calculatoarelor),.

11 C# and JS - OOP

- **C#:**
  - Similar cu Java, derivat din C++.
  - Folosește `namespace` pentru organizare.
  - Moștenire simplă (`class Triangle : Shape2D`).
  - Metoda `Main` este punctul de intrare.
- **JavaScript:**
  - Bazat pe prototipuri (deși are sintaxa `class` în ES6).
  - Obiectele sunt colecții de proprietăți și metode.
  - `class Person { constructor(n) { this.name = n; } }`.
  - Câmpuri private cu `#` (`#balance`).

Rezumat

Iată o descriere detaliată a **Cursului 11** ("Object-Oriented Programming Languages 2"), structurată pe cele trei limbaje majore analizate în document: C#, Python și JavaScript. Acest curs continuă studiul paradigmei OOP, concentrându-se pe sintaxa și particularitățile implementării conceptelor fundamentale (moștenire, încapsulare, polimorfism) în aceste limbaje moderne.

1. Programare Orientată pe Obiecte în C#

Această secțiune prezintă C# ca un limbaj derivat din C/C++ și similar cu Java, construit de Microsoft pentru platforma .NET.

- **Arhitectură și Caracteristici:**
  - Se bazează pe **MSIL** (Microsoft Intermediate Language) și rulează pe mașina virtuală .NET Framework, asigurând portabilitate (concept preluat de la Java).

    - Permite integrarea completă cu platforma Windows și scrierea programelor în mai multe limbaje care compilează în același MSIL.
    - Punctul de intrare în program este funcția `Main`.
- **Moștenirea în C#:**
    - Clasele sunt organizate ierarhic: **Superclasă** (bază) și **Subclasă** (derivată).
    - **Nu** este permisă moștenirea multiplă (o clasă nu poate avea mai multe superclase).
    - *Sintaxă:* Se folosește simbolul `:` pentru derivare. Exemplu: `class Triangle : Shape2D`.
- **Exemplu de Implementare (Forme Geometrice):**
    - Clasa de bază `Shape2D` definește atribute generice (`width`, `height`) și metoda `showDim()`.
    - Clasa derivată `Triangle` adaugă funcționalitate specifică (`typeOfTriangle`, metoda `area()`) și refolosește atributele din `Shape2D`.
    - În `Main`, obiectele sunt instanțiate cu `new Triangle()`, iar accesul la membri se face prin operatorul punct (`.`).

2. Programare Orientată pe Obiecte în Python

Python este prezentat ca un limbaj care implementează conceptele OOP într-un stil specific, compact și dinamic.

- **Clase și Obiecte:**
    - Definirea clasei se face cu `class Nume:`.
    - Constructorul este metoda specială `__init__(self, ...)`. Parametrul `self` este obligatoriu și referă instanța curentă.
    - Atributele de instanță sunt definite în constructor prin `self.nume_atribut`.
- **Pilonii OOP în Python:**
    - **Încapsulare:** Se realizează prin convenție, folosind prefixul `__` (dublu underscore) pentru a face membrii privați (ex: `self.__balance`). Accesul se face prin metode publice (getters/setters).
    - **Moștenire:** Clasa derivată specifică părintele în paranteze: `class Car(Vehicle):`. Constructorul părintelui se apelează cu `super().__init__(...)`.
    - **Polimorfism:** Metode cu același nume (`area`) se comportă diferit în clase diferite (`Circle`, `Square`).
    - **Abstractizare:** Se folosește modulul `abc` (`ABC`, `abstractmethod`) pentru a defini clase care nu pot fi instanțiate și impun implementarea anumitor metode.
- **Funcționalități Avansate:**
    - **Metode Statice:** Decorate cu `@staticmethod`, nu primesc `self`.
    - **Metode de Clasă:** Decorate cu `@classmethod`, primesc `cls` (referința la clasă).
    - **Dunder Methods (Magic Methods):** Metode speciale cu `__` care permit suprascrierea comportamentului operatorilor sau funcțiilor interne (ex: `__str__` pentru reprezentarea ca șir).

3. Programare Orientată pe Obiecte în JavaScript

Această secțiune explică tranziția JavaScript de la un limbaj bazat pe prototipuri la unul care suportă sintaxa modernă de clase (ES6+).

- **Crearea Obiectelor:**
  - **Literal:** `let person = { name: "John", greet: function() {...} }`.
  - **Clase (Sintaxa ES6):** `class Person { constructor(...) { ... } }`.
- **Membri și Acces:**
  - Proprietățile se accesează prin `.` sau `[]` (ex: `person['age']`).
  - **Metode:** Se definesc direct în corpul clasei, fără cuvântul cheie `function`.
- **Particularități OOP:**
  - **Încapsulare (Câmpuri Private):** Se folosește prefixul `#` pentru a defini câmpuri private (ex: `#balance`), care nu sunt accesibile din exterior.
  - **Moștenire:** Se folosește cuvântul cheie `extends` (ex: `class Dog extends Animal`).
  - **Polimorfism:** Metodele din clasa copil pot avea același nume cu cele din părinte, dar comportament diferit.
  - **Abstractizare:** Deși JS nu are clase abstracte native ca C# sau Java, conceptul se simulează aruncând o eroare (`throw new Error`) în metoda din clasa de bază dacă aceasta nu este suprascrisă.
  - **Getters și Setters:** Se folosesc cuvintele cheie `get` și `set` pentru a defini proprietăți calculate (ex: `get area() { return ... }`).
  - **Metode Statice:** Definite cu `static`, apelabile direct pe clasă (`MathUtils.add(2,3)`), nu pe instanță.

4. Bibliografie

Cursul se încheie cu referințe bibliografice clasice pentru studiul limbajelor de programare, incluzând autori precum Horia Ciocârlie, Carlo Ghezzi și Donald Knuth.

12 How to trigger an exception

- **Ada:** `raise error;` (unde `error` este o excepție definită).
- **C#:** `throw new DivideByZeroException();` într-un bloc `try`.
- **Python:** Deși sursele se concentrează pe `try-except`, declanșarea se face implicit la erori (ex: `10/0`) sau explicit cu `raise` (menționat contextual la gestionarea erorilor).

Rezumat

Iată o descriere detaliată a **Cursului 12** ("Structuri de Control"), structurată pe capitolele principale din documentul sursă. Acest curs analizează mecanismele prin care programatorul controlează fluxul execuției, de la instrucțiuni simple până la modularizare complexă.

1. Structuri de Control la Nivel de Instrucțiune

Această secțiune clasifică instrucțiunile care specifică ordinea execuției într-un program individual.

- **Secvența:**
  - Este cea mai simplă structură (specifică limbajelor imperative), unde instrucțiunile se execută în ordinea scrierii.
  - Se implementează prin blocuri de cod: `{ ... }` în C/Java/C# sau indentare în Python.
- **Selecția (Decizia):**
  - Permite alegerea unei alternative bazată pe o condiție logică.
  - **If-Then-Else:** Prezentat cu sintaxa specifică Pascal (`if ... then ... else`) și Python (`if ... elif ... else`).
  - **Selecția Multiplă:**
    - Utilizează `case` (Pascal, Ada) sau `switch` (C, Java, C#).
    - Include mecanisme pentru cazul implicit ("default"): `others` în Ada, `default` în C, `else` în Pascal.
    - *Notă Python:* Python nu are `switch` nativ tradițional; cursul exemplifică emularea acestuia folosind un dicționar (mapare cheie-valoare).
- **Repetiția (Buclele):**
  - **Controlate de condiții:**
    - Test inițial: `while` (se execută de 0 sau mai multe ori).
    - Test final: `repeat ... until` (Pascal) sau `do ... while` (C/Java) - se execută cel puțin o dată.
  - **Controlate de contoare:**
    - Instrucțiunea `for`. În C/Java este flexibilă (`for(init; cond; step)`), în timp ce în Pascal/Algol este mai restrictivă (`to` , `by`).
  - **Ieșirea din bucle:** Se face prin `break` , `continue` (în C/Java), `exit` (în Ada - permite ieșirea din bucle etichetate) sau `goto` .

2. Subprograme: Probleme Specifice

Cursul abordează problemele subtile care apar la apelul funcțiilor și procedurilor.

- **Efecte Colaterale (Side Effects):**
  - Apar când un subprogram modifică o entitate care nu este locală (ex: o variabilă globală). Acest lucru face expresiile de tipul `v = a + f(a,b) + c` impredictibile dacă `f` modifică `a` .
- **Pseudonime (Aliasing):**
  - Situația în care aceeași locație de memorie este accesibilă prin două nume diferite simultan.
  - **Cauze majore (exemplificate în C++):**

1. Transmiterea prin referință a unei variabile globale care este folosită și direct în funcție.
2. Transmiterea aceleiași variabile pentru doi parametri diferiți transmiși prin referință (`void p(int &x, int &y)` apelat cu `p(z, z)`).
3. Vectori: `t[i]` și `t[j]` sunt pseudonime dacă `i == j` .

3. Tratarea Excepțiilor

Sunt prezentate mecanismele de gestionare a erorilor la execuție în trei limbaje diferite.

- **Ada:**
  - Excepții predefinite (ex: `constraint_error` , `storage_error` , `numeric_error`).
  - Mecanism: Declarare (`error: exception`), Declanșare (`raise error`), Tratare (`when error => ...`).
  - Exemplu detaliat: Implementarea unei Stive (Stack) care aruncă erori la `push` (dacă e plină) sau `pop` (dacă e goală).
- **C#:**
  - Toate excepțiile sunt clase derivate din `System.Exception` .
  - Clase comune: `DivideByZeroException` , `IndexOutOfRangeException` , `NullReferenceException` .
  - Structura: `try` (cod monitorizat) -> `catch` (tratare) -> `finally` (cod executat obligatoriu).
- **Python:**
  - Folosește blocuri `try-except-finally` .
  - Introduce contextul `with` pentru gestionarea resurselor (ex: fișiere), care închide automat resursa chiar și în caz de eroare, eliminând nevoia de `finally` explicit pentru `close()` .

4. Modularizare: Pachete și Namespace-uri

Organizarea codului pentru proiecte mari.

- **Pachete în Java:**
  - Un pachet grupează clase conexe. Se definește cu `package nume;` și se utilizează cu `import` .
  - Pachete standard esențiale: `java.lang` (importat automat, conține `String` , `Math`), `java.util` (colecții), `java.io` .
  - Controlul accesului: `public` (vizibil oriunde) vs. `default` (vizibil doar în pachet).
- **Namespace-uri în C#:**
  - Containere logice pentru clase, similare pachetelor, declarate cu `namespace` .
  - Rezolvă conflictele de nume (două clase cu același nume pot exista în namespace-uri diferite).

- Permit folosirea **alias-urilor** (`using ProjA = ProjectA.Namespace1`) pentru simplificare.

5. Module Reutilizabile (Java vs C#)

Nivelul superior de modularizare pentru distribuția codului.

- **Module în Java (din versiunile noi):**
  - Definite în fișierul `module-info.java`.
  - Folosesc directivele `exports` (ce pachete sunt vizibile altora) și `requires` (de ce alte module depinde).
- **Ansamble (Assemblies) în C#:**
  - Unități de compilare (`.dll` pentru biblioteci, `.exe` pentru executabile).
  - Permit reutilizarea codului între proiecte prin referințe.
  - Exemplu practic: Crearea unui `Class Library` cu funcții matematice (`MathUtilities.dll`) și utilizarea lui în alt proiect prin `using`.
  - Testarea se face folosind framework-uri precum NUnit sau xUnit.

13 - Reduction of Expression (Lambda Calculus)

- Trebuie să aplici **Beta-reducerea**: înlocuirea variabilei legate cu argumentul aplicat.
  - *Exemplu:* `(λx.x) a` -> `a`.
  - *Exemplu Logic:* `AND true false`.
    - Definiții: `true = λx.λy.x`, `false = λx.λy.y`.
    - `and = λp.λq.p q p` (sau forma cu `cond`: `λx.λy.(cond y false x)`).
    - Calcul: `(and true false)` -> `true false true` -> `(λx.λy.x) false true` -> `false`.

Rezumat

Iată o descriere detaliată a **Cursului 13** ("Fundamentele programării funcționale"), structurată pe capitolele principale prezentate în sursă. Acest curs introduce paradigma funcțională, punând accent pe matematica din spatele acesteia (Calculul Lambda) și pe proprietățile teoretice ale evaluării expresiilor.

1. Introducere și Transparența Referențială

Cursul începe prin definirea programării funcționale ca un stil bazat pe evaluarea funcțiilor matematice, eliminând efectele secundare.

- **Transparența Referențială:** Este proprietatea centrală a limbajelor funcționale pure.
  - *Definiție:* Semantica (și valoarea) unei expresii depinde exclusiv de semantica componentelor sale, nu de istoricul execuției sau de starea globală.
  - *Exemplu:* Într-o expresie `(f + g) * (x + y)`, calculul `f + g` este independent de

`x + y`.
    - *Consecințe:* Absența variabilelor globale și a atribuirilor distructive permite verificarea formală ușoară și execuția paralelă, deoarece ordinea evaluării nu schimbă rezultatul.
- **Variabile și Atribuire:**
    - În limbajele imperative (nefuncționale), variabilele și atribuirea introduc dependența de istoric ("state"), ceea ce distruge transparența referențială. Dacă o funcție modifică o variabilă globală, apeluri identice pot returna valori diferite.

2. Calculul Lambda (Lambda Calculus)

Dezvoltat de Alonzo Church în anii '30, este un sistem formal care stă la baza tuturor limbajelor funcționale.

- **Expresia Lambda (λ):**
    - Forma canonică: $\lambda nume.\, corp$.
    - *Componente:* `nume` (după λ) este **variabila legată** (parametrul formal), iar `corp` este expresia returnată.
    - *Aplicarea funcției:* Se scrie `(funcție argument)`. Funcția nu are nume, fiind anonimă.
- **Funcții fundamentale:**
    - **Funcția Identitate:** $\lambda x.\, x$ (returnează argumentul primit).
    - **Auto-aplicarea:** $\lambda a.\, (a\ a)$ (aplică argumentul asupra lui însuși). Dacă se aplică auto-aplicarea asupra ei însăși, rezultă o buclă infinită.

3. Reducerea Beta (-reduction) și Legarea Variabilelor

Acesta este mecanismul efectiv de "execuție" sau evaluare în calculul lambda.

- **Reducerea Beta:** Procesul de înlocuire a tuturor aparițiilor variabilei legate din corpul funcției cu argumentul furnizat.
    - Exemplu: `(λx.x) a` devine `a`.
- **Variabile Libere vs. Legate (Free vs. Bound):**
    - O variabilă este **legată** dacă apare în corpul unei funcții care o declară ca parametru ($\lambda.\ldots\ldots$).
    - O variabilă este **liberă** dacă nu este definită în contextul curent (ex: `v` în expresia `(a b v)` este liberă).
- **Domeniul de vizibilitate:** Domeniul unei variabile legate se extinde asupra corpului funcției, dar poate fi "umbrit" de o altă funcție interioară care declară un parametru cu același nume.

4. Conflicte de Nume și Conversia Alfa (-conversion)

Atunci când aplicăm reducerea Beta, pot apărea conflicte dacă un argument conține variabile libere care, prin înlocuire, devin accidental legate (capturate) de o funcție interioară.

- **Problema:** Dacă `f = λx.λy.(x y)` și aplicăm `f y z`, variabila liberă `y` din argument ar putea fi capturată de `λy` din interiorul lui `f`, schimbând sensul programului.
- **Soluția (Conversia Alfa):** Redenumirea variabilei legate și a tuturor apariițiilor sale într-un nume nou, nefolosit (ex: din $\lambda.(\ldots)$ în $\lambda.(\ldots)$), înainte de substituție.

5. Reducerea Mu (-reduction)

Este o regulă de simplificare (optimizare) a expresiilor lambda.

- **Regula:** $\lambda nume.(expree\ nume)$ este echivalent cu `expresie`, dacă `nume` nu apare liber în `expresie`. Practic, elimină o încapsulare redundantă a unei funcții.

6. Valori Logice și Operatori (Applied Lambda Calculus)

Cursul demonstrează cum se pot modela conceptele booleene folosind doar funcții, fără tipuri de date primitive.

- **Valori de adevăr:**
  - **TRUE** este definit ca funcția care selectează primul argument: $\lambda p.\lambda.p$ (similar cu `sel first`).
  - **FALSE** este definit ca funcția care selectează al doilea argument: $\lambda p.\lambda.$ (similar cu `sel second`).
- **Expresia Condițională (`cond`):**
  - Modelată ca $\lambda e.\lambda e.\lambda c.(c\ e\ e)$. Practic, condiția `c` este aplicată asupra celor două opțiuni. Dacă `c` este `true`, alege `e1`; dacă e `false`, alege `e2`.
- **Operatori Logici:**
  - **NOT:** $\lambda x.(con\ ae\ rue\ x)$ — inversează selecția.
  - **AND:** $\lambda x.\lambda.(con\ \ ae\ x)$ — dacă `x` e adevărat returnează `y`, altfel `false`.
  - **OR:** $\lambda x.\lambda.(con\ rue\ \ x)$ — dacă `x` e adevărat returnează `true`, altfel `y`.

7. Bibliografie

Referințele includ lucrări clasice precum "The Art of Computer Programming" (Knuth) și cursuri universitare de la UPT (Horia Ciocârlie).

14 Write comprehension, memory efficient, parenthesis

- Se referă la **Python Generator Comprehensions**.
- **Sintaxă:** Folosește paranteze rotunde `()` în loc de pătrate `[]`.
  - Exemplu: `gen = (x**2 for x in range(5))`,.

- **Memory Efficient:** Este "lazy" (leneșă) - generează valorile una câte una la cerere (`next()`), nu încarcă toată lista în memorie ca List Comprehension.

Rezumat

Iată explicarea detaliată a **Cursului 14** („Programarea funcțională în practică”), bazată pe noile surse furnizate. Acest curs este foarte important pentru subiectele practice de examen, în special pentru **Python Comprehensions** și **Java Streams**.

Cursul se concentrează pe tehnici de scriere a codului „curat” (clean code) prin aplicarea principiilor funcționale în limbaje moderne (Python și Java).

1. Evitarea Controlului Fluxului (Avoiding Flow Control)

Obiectivul este trecerea de la stilul imperativ (concentrat pe **CUM** se obțin datele, folosind bucle explicite și variabile de stare modificabile) la cel funcțional (concentrat pe **CE** sunt datele),.

- **Problema:** Codul imperativ cu bucle `for/while` implică variabile de stare (`state_var`) care se modifică în timp (efecte secundare), ceea ce face codul greu de urmărit.
- **Soluția:** Încapsularea logicii de construcție a datelor în funcții separate. În loc să scrii o buclă uriașă care construiește o colecție, definești o funcție `make_collection(data)` care returnează direct rezultatul dorit,.

2. Comprehensions (Esențial pentru Subiectul 14 din lista ta)

Acestea sunt expresii compacte care înlocuiesc buclele de construcție a listelor/dicționarelor.

- **List Comprehensions:** `[x for x in data if cond]` . Sunt "eager" (evaluează totul imediat) și ocupă memorie pentru toată lista.
- **Generator Comprehensions (Memory Efficient):**
  - **Sintaxă:** Folosesc paranteze rotunde `()` în loc de paranteze drepte `[]` .
  - **Caracteristici:** Sunt **lazy** (leneșe). Datele nu sunt generate toate odată, ci una câte una, doar când sunt cerute (prin `.next()` sau iterare).
  - **Avantaj:** Sunt foarte **eficiente cu memoria** pentru seturi mari de date sau secvențe infinite, deoarece nu încarcă totul în RAM,.
  - *Exemplu cod:* `gen = (x**2 for x in range(5))` .
- **Set & Dictionary Comprehensions:**
  - Set: `{chr(65+i) for i in range(6)}` -> `{'A', 'B', ...}` .
  - Dict: `{i:chr(65+i) for i in range(6)}` -> `{0: 'A', 1: 'B', ...}` .

3. Recursivitate (Recursion)

Programarea funcțională încurajează recursivitatea în locul iterației pentru a evita starea mutabilă.

- **Limitări în Python:** Python nu este optimizat pentru recursivitate adâncă (nu are *tail call elimination* - eliminarea apelului pe coadă). Limita implicită a stivei este 1000.
- **Exemplu Quicksort funcțional:**

```
def quicksort(lst):
    if len(lst) == 0: return lst
    pivot = lst
    # Foloseste list comprehensions pentru partitionare
    small = quicksort([x for x in lst if x < pivot])
    large = quicksort([x for x in lst if x > pivot])
    return small + [x for x in lst if x == pivot] + large
```

.

4. Eliminarea Buclelor (Eliminating Loops)

Metoda sistematică de a elimina buclele este folosirea funcțiilor de ordin superior (High Order Functions) precum `map`, `filter`, `reduce`.

- În loc de:

```
for e in it: func(e)
```

- Se folosește:

```
map(func, it)
```

  Aceasta elimină legarea repetată a variabilei de iterație.

5. Java Streams (Fluxuri în Java)

Acesta este un capitol masiv în curs, relevant pentru subiectele de cod Java. Stream-urile (introduse în Java 8) sunt "wrappers" peste surse de date care permit procesarea funcțională fără a modifica sursa originală.

Operații Cheie pe Stream-uri:

1. **Creare:** `Stream.of(array)` sau `lista.stream()`.
2. `map` **(Intermediar):** Transformă fiecare element din stream aplicând o funcție.
   - *Exemplu:* `stream.map(employeeRepository::findById)` transformă un ID în obiectul Angajat,.
3. `filter` **(Intermediar):** Păstrează doar elementele care satisfac un predicat (condiție).
   - *Exemplu:* `.filter(e -> e.getSalary() > 200000)`,.
4. `collect` **(Terminal):** "Împachetează" rezultatul procesării într-o colecție (Listă, Set etc.).
   - *Exemplu:* `.collect(Collectors.toList())`.
5. `findFirst` **(Terminal):** Returnează un obiect `Optional` (care poate fi null) cu primul element găsit.

6. `flatMap`: "Aplatizează" structuri complexe. Dacă ai o listă de liste, `flatMap` o transformă într-un singur stream continuu de elemente.
   - *Exemplu:* `Stream<List<String>>` devine `Stream<String>`.
7. `peek` **(Intermediar vs Terminal):**
   - Spre deosebire de `forEach` (care e terminal și încheie stream-ul), `peek` permite executarea unei acțiuni pe fiecare element (ex: printare, modificare stare internă obiect) și **returnează stream-ul mai departe** pentru alte procesări.
   - *Exemplu:* `.peek(e -> e.salaryIncrement(10.0))`.

Sinteză pentru Examen (Subiectele tale):

- **Pentru Subiectul 14 (Comprehension, memory efficient):** Trebuie să scrii un **Generator Comprehension** în Python.
  - *Cod:* `g = (x * 2 for x in range(1000000))`
  - *Explicație:* Folosește paranteze rotunde, este "lazy" (nu alocă memorie pentru 1 milion de elemente, ci le generează la cerere), spre deosebire de lista `[...]`.
- **Pentru Subiectul 1 (Piece of code):** Poți primi un snippet de **Java Stream** și să fii întrebat ce face (ex: un lanț `filter` -> `map` -> `collect`).
- **Pentru Subiectul 4 (Eliminating loops):** Să transformi un `for` simplu într-un `map` sau o recursivitate.

Extra: From C# - link features & Design Pattern Code

- **Link features from C#:**
  - **Namespaces:** Organizarea logică a codului (`namespace MyApp`), import cu `using`.
  - **Assemblies:** Unități de compilare (`.dll`), reutilizabile.
  - **Acces:** `public`, `private`, `protected`, `internal`.
- **Code for a Design Pattern:**
  - Singurul "pattern" arhitectural cu cod complet în cursuri este **Client-Server** (Java) din Curs 02.
  - Codul implică o clasă `Server` care face `new ServerSocket(port)` și `accept()`, și o clasă `Client` care face `new Socket(ip, port)`.

Sfaturi pentru examen: Focusează-te pe sintaxa specifică din exemplele cursurilor (Lambda calcul, codul de server Java, pointeri C, moștenire C++/Java) și pe diferențele conceptuale (static vs dinamic scoping, interpretare vs compilare).